
Table of Contents

.....	1
Load Displacement Measurements	1
Define Physical Constants	1
Continuous-Time State-Space Model	1
Discretization using Zero-Order Hold (ZOH)	2
Kalman Filter Parameters	2
Kalman Filter Implementation	2
Trajectory Plot	3
State Tracking Plots	3

```
clc;
clear;
close all;
```

Load Displacement Measurements

```
data = load('HW1_Q1_Displacement_Measurements.txt');

t = data(:,1);
ux_meas = data(:,2);
uy_meas = data(:,3);

N = length(t);
dt = t(2) - t(1); % Sampling time (100 fps → dt = 0.01 s)
```

Define Physical Constants

```
g = 9.81; % Gravity (m/s^2)
```

Continuous-Time State-Space Model

```
Ac = [ 0  1  0  0;
       0  0  0  0;
       0  0  0  1;
       0  0  0  0 ];

Bc = [ 0;
       0;
       0;
       -g ];

Cc = [ 1  0  0  0;
       0  0  1  0 ];

Dc = zeros(2,1);
```

```
% Create continuous state-space system
sysc = ss(Ac, Bc, Cc, Dc);
```

Discretization using Zero-Order Hold (ZOH)

```
sysd = c2d(sysc, dt, "zoh");
```

```
Ad = sysd.A;
Bd = sysd.B;
Hd = sysd.C;
```

Kalman Filter Parameters

```
R = [100 0;
      0 100];           % Measurement noise covariance

Q = zeros(4);           % No process noise

% Initial State
X0 = [300; 100; 200; 60];

% Two cases of initial covariance
P_cases = {eye(4)*1e4, eye(4)*1e8};
```

Kalman Filter Implementation

```
estimates = cell(2,1);

for caseID = 1:2

    P = P_cases{caseID};
    X = X0;

    X_est = zeros(4,N);

    for k = 1:N

        % Prediction
        X_pred = Ad*X + Bd;
        P_pred = Ad*P*Ad' + Q;

        % Measurement vector
        z = [ux_meas(k); uy_meas(k)];

        % Kalman Gain
        K = P_pred*Hd' / (Hd*P_pred*Hd' + R);

        % Update
        X = X_pred + K*(z - Hd*X_pred);
        P = (eye(4) - K*Hd)*P_pred;

        X_est(:,k) = X;
```

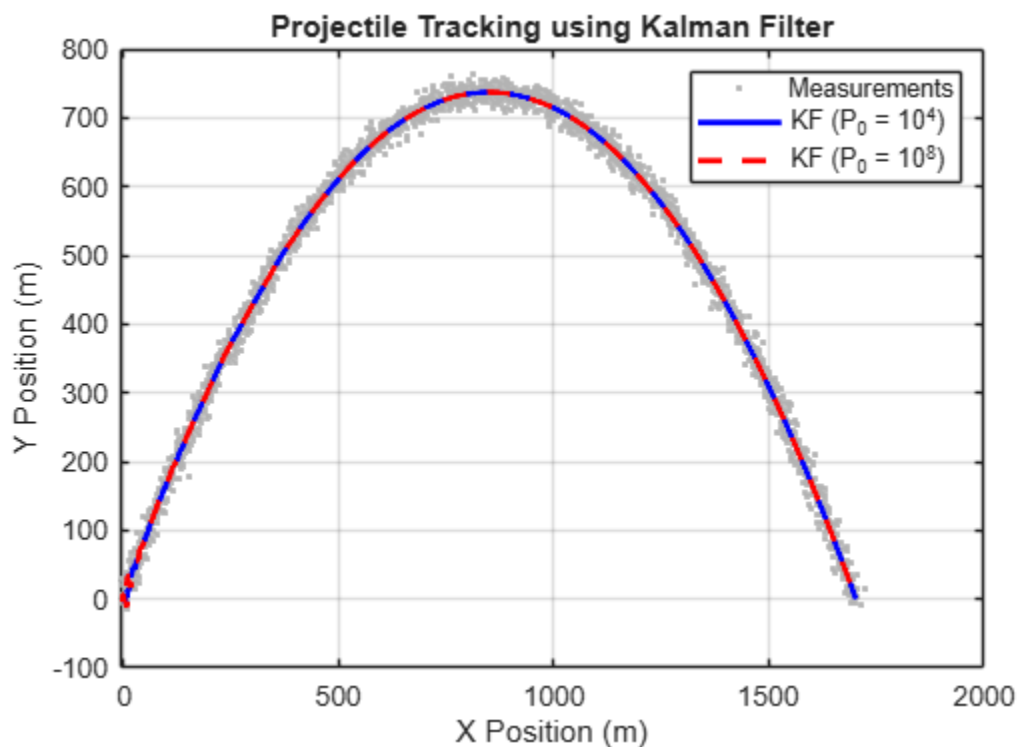
```
end
```

```
estimates{caseID} = X_est;
```

```
end
```

Trajectory Plot

```
figure;  
plot(ux_meas, uy_meas, '.', 'Color',[0.7 0.7 0.7]); hold on;  
plot(estimates{1}(1,:), estimates{1}(3,:), 'b', 'LineWidth',2);  
plot(estimates{2}(1,:), estimates{2}(3,:), 'r--', 'LineWidth',2);  
  
legend('Measurements', 'KF ( $P_0 = 10^4$ )', 'KF ( $P_0 = 10^8$ )');  
xlabel('X Position (m)');  
ylabel('Y Position (m)');  
title('Projectile Tracking using Kalman Filter');  
grid on;
```



State Tracking Plots

```
stateNames = {'u_x', 'v_x', 'u_y', 'v_y'};
```

```
for i = 1:4
```

```
    figure;  
    plot(t, estimates{1}(i,:), 'b', 'LineWidth',1.8); hold on;
```

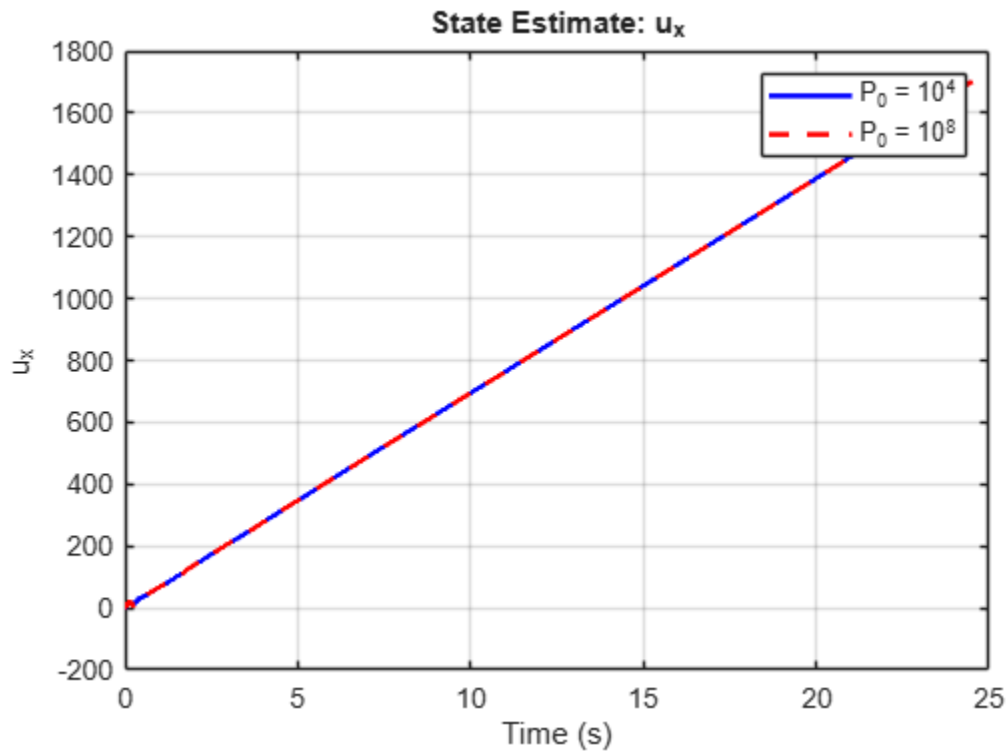
```

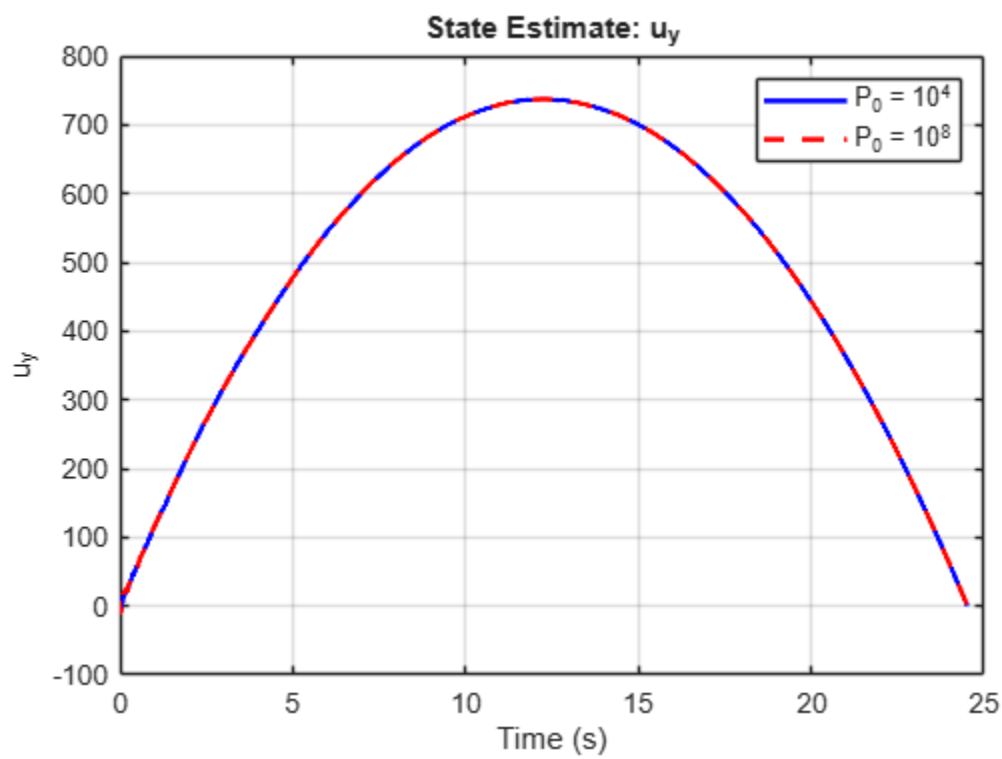
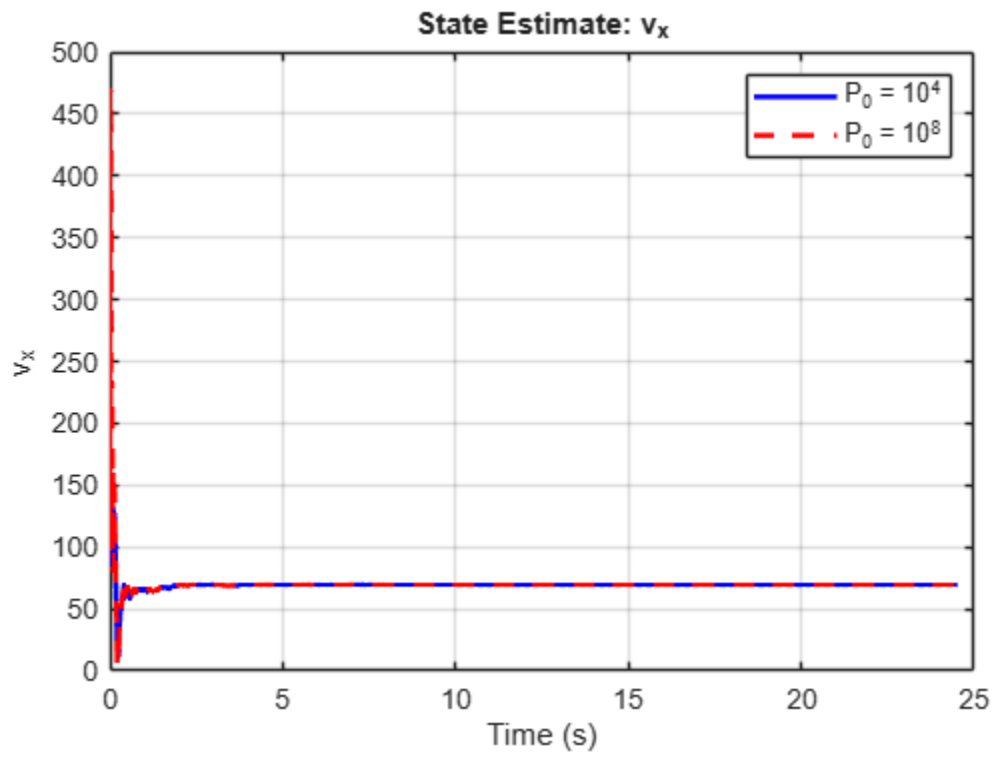
plot(t, estimates{2}(i,:), 'r--', 'LineWidth', 1.8);

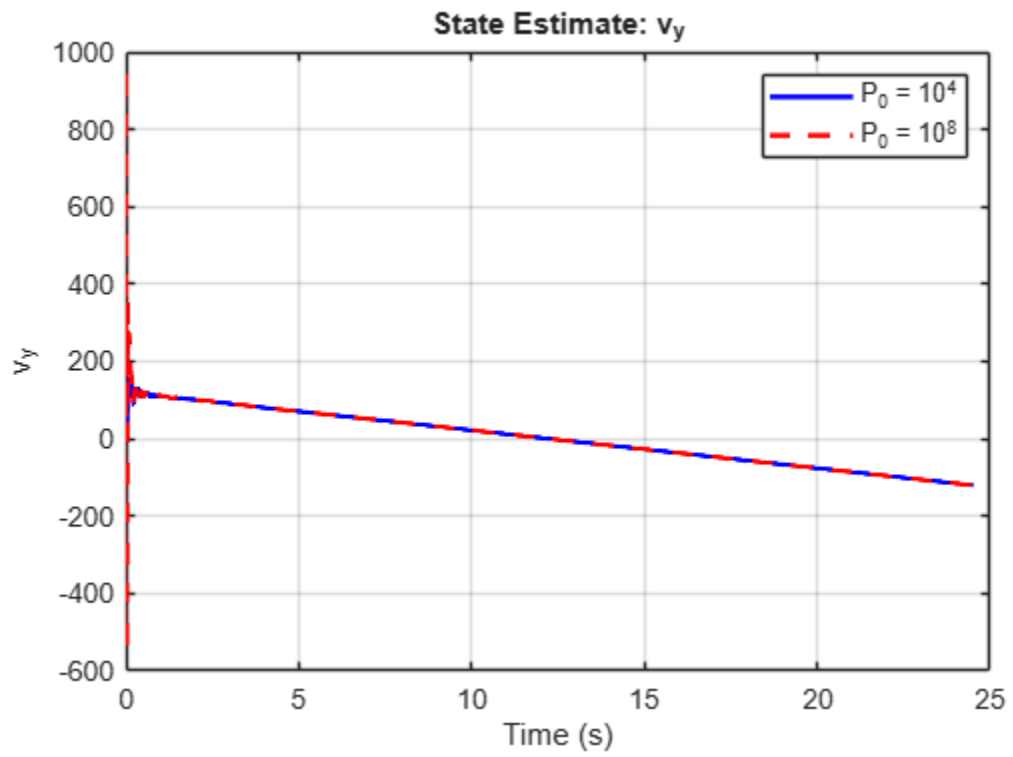
xlabel('Time (s)');
ylabel(stateNames{i});
legend('P_0 = 10^4', 'P_0 = 10^8');
title(['State Estimate: ', stateNames{i}]);
grid on;

```

```
end
```







Published with MATLAB® R2025b